



Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure, Environment, Actuators, and Sensors.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

The variables representing the physical environment state are ``self.width``, ``self.height``, ``self.walls``, ``self.food_positions``, ``self.opponents``, and the newly added ``self.toxic_traps``.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

It is a "Multi-Agent" environment. The presence of ``self.opponents`` means there are other entities taking actions in the environment that actively change its state and can interact with (or penalize) the main agent.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of everything the agent has perceived (received via its sensors from the environment) up to the current moment in time.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

It represents the Sensors (S) component, as it gathers data from the environment state to pass to the agent.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

It is "Partially Observable". While the agent knows its own position and the opponents' positions exactly, it does not receive the global coordinates of the walls or the food. It only receives boolean flags (``smells_food``, ``hit_wall``) for its current local position and the total count of ``remaining_food``.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance Measure (P) component (represented by ``self.score``).

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Because AI engineering is focused on "Acting Rationally" (results-oriented behavior maximizing utility) rather than "Thinking Humanly" (internal cognitive effort). As highlighted by the Aeronautics principle in the lecture, success is measured by the external objective (e.g., successful navigation without crashing) rather than the internal effort exerted to calculate the move.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples

safely avoiding position $(0, 0)$, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

It strictly enforces or increases the environment's "Partial Observability". The agent lacks critical state information (the locations of the traps) necessary to make fully informed, optimal decisions, forcing it to act under uncertainty.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos)` in `self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

A rational agent acts to achieve the best possible expected outcome. By giving the agent a sensor to detect a toxin (`'smells_toxin'`), the agent can use this percept to adjust its future actions (e.g., moving away from the trap) to avoid the -15 point penalty, thereby maximizing its performance measure (score).

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example

discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic exploit occurs when a vacuum agent is rewarded for "cleaning up dirt" rather than "having a clean floor." A flawed rational agent will learn to dump dirt back onto the floor just to clean it up again and endlessly farm points, demonstrating the danger of poorly designed performance metrics.



Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!



FACULTY OF COMPUTING